Students Ergonomics Monitoring

Posture Module — Feature Extraction Setup

Laboratory of Nearable Technologies for Health Data Science
Politecnico di Milano — A.Y. 2025/2026

May 2026

Project Overview

The goal of this project is to build a machine learning system that classifies a student's ergonomic state as **Normal** or **Anomalous** while studying at a laptop. The system combines three data sources captured simultaneously:

- **Posture** — webcam-based pose estimation (MediaPipe Pose)
- **Blinking** — webcam-based eye blink detection (MediaPipe FaceMesh)
- **Heart Rate** — wearable sensor

Two ML models are trained on the combined data: a **supervised** classifier (using labeled Normal/Anomalous data) and an **unsupervised** anomaly detector (learning the normal distribution and flagging deviations). Each module independently extracts its own feature columns, which are then merged horizontally into a single training dataset in the `SensorFusion` module.

Data Pipeline

Pre-recorded videos are organized into two folders: `Video/Normal/` and `Video/Anomalous/`. Each video is approximately 10 minutes long.

A **sliding window** approach is used to extract features:

- **Window size:** 60 seconds
- **Step:** 10 seconds
- **Overlap:** 50 seconds between consecutive windows

This produces approximately 55 rows per 10-minute video. Each module processes the same videos and produces a CSV with the same number of rows, aligned by `window_start` timestamp. The final training dataset is obtained by horizontally concatenating the posture, blinking, and heart rate CSVs.

Output files:

```
data/raw/posture_normal.csv -- features from Normal videos
data/raw/posture_anomalous.csv -- features from Anomalous videos
```

Posture Parameters

The posture module uses **MediaPipe Pose** (BlazePose, 33 keypoints) to extract body landmarks from each video frame. Three postural parameters are computed per frame:

Thresholds are derived from scientific literature (Scoliometer – ScienceDirect, Hustinawaty 2025, BMJ Open 2016 CUELA norms, PostureCV). They are used to compute the `bad_pct` feature:

Parameter	Computation	Threshold
Shoulder Asymmetry	Angle of the line connecting left and right shoulder (landmarks 11, 12) relative to the horizontal axis, computed with `atan2`. A value of 0 means perfectly level shoulders.	7
Head Lateral Tilt	Angle between the ear midpoint (landmarks 7, 8) → shoulder midpoint (landmarks 11, 12) line and the vertical axis. A value of 0 means the head is centered above the shoulders.	10
Forward Head Proxy	Z-depth difference between the nose (landmark 0) and the shoulder midpoint (landmarks 11, 12), multiplied by 100. Captures how far forward the head is relative to the torso.	15

Table 1: Postural parameters and their literature-based thresholds.

the percentage of frames within a window where the absolute value of a parameter exceeds its threshold.

Extracted Features (9 columns)

For each 60-second window, the following 9 features are computed:

Column	Description
`p_shoulder_mean`	Mean of $\|shoulderasymmetry\|$ across all frames in the window
`p_shoulder_std`	Standard deviation of $\|shoulderasymmetry\|$ in the window
`p_shoulder_bad_pct`	Percentage of frames where $\|shoulderasymmetry\| > 7$
`p_head_tilt_mean`	Mean of $\|headlateraltilt\|$ across all frames in the window
`p_head_tilt_std`	Standard deviation of $\|headlateraltilt\|$ in the window
`p_head_tilt_bad_pct`	Percentage of frames where $\|headlateraltilt\| > 10$
`p_fhp_mean`	Mean of $\|forwardheadproxy\|$ across all frames in the window
`p_fhp_std`	Standard deviation of $\|forwardheadproxy\|$ in the window
`p_fhp_bad_pct`	Percentage of frames where $\|forwardheadproxy\| > 15$

Table 2: Feature columns produced by the posture module for each sliding window.

The **mean** captures average posture quality. The **standard deviation** captures postural instability (frequent shifting). The **bad_pct** captures how often the parameter was in a problematic range. Together, these give the downstream ML model a rich description of the student's posture over each window.

Code Structure

How to Run

Place video files in the appropriate folders, then run from the `Scripts/Posture/` directory:

```
python main.py # process both Normal and Anomalous
python main.py --normal # process only Normal videos
python main.py --anomalous # process only Anomalous videos
```

File	Role
`config.py`	All configuration: MediaPipe settings, thresholds, window parameters, file paths.
`pose_extractor.py`	Opens a video file and iterates over frames, extracting 33 MediaPipe Pose landmarks per frame.
`posture_analyzer.py`	Takes landmarks from a single frame and computes the 3 raw numerical values.
`feature_extractor.py`	Accumulates per-frame values, then computes the 9 features for each sliding window.
`main.py`	Entry point. Scans video folders, processes all videos, saves output CSVs.

Table 3: Files in `Scripts/Posture/`.

The script overwrites the output CSVs each time, using all videos found in each folder. To expand the dataset, add more videos and re-run.

Next Steps

Once the blinking and heart rate modules produce their own CSVs with the same windowing scheme, the `SensorFusion` module will merge all columns by `window_start` and train the supervised and unsupervised models on the combined feature set.

For live deployment, the same feature extraction logic will run on the webcam feed with a rolling 60-second buffer, emitting a feature row every 10 seconds for real-time classification.